

COMP3330/6380, Semester 1, 2026

Assessment 1 (Group Project): Deep Learning for a Smart Fridge

Assessment Type	Project
Weighting	10% for COMP3330 and 10% for COMP6380
Length	COMP3330 groups: 6-10 pages excluding References/Appendix COMP6380 groups: 8-14 pages excluding References/Appendix
Due Date	23 April 2026 (9 am)
Submission	Online via Canvas / Turnitin

Description

Student groups work to find solutions to deep learning tasks by using Python, PyTorch, and other relevant Python packages. The submission includes:

1. A group report that addresses the questions specified in this document with written text, figures, and tables. The report is to be formatted in Springer LNCS format and exported to pdf format. The LNCS conference paper template is available at the following link: <https://www.springer.com/gp/computer-science/lncs/conference-proceedings-guidelines>. The report length in pages should fall in the given range. To save space, figures and/or tables can be placed into an Appendix.
2. A trained model file for image classification (see Question 1).
3. A training script or notebook used to train the submitted model, a Python inference script named `inference.py`, and any other scripts used in the pipeline (e.g., data processing), if relevant.
4. A README file explaining how to run the submitted code.
5. (If necessary) a `requirements.txt` file specifying packages required to run the submitted code.
6. A document that states the contributions of each group member. If any group management issues arise please inform teaching staff.
7. All code used to produce the results discussed in the report as Supplemental Material. Marks may be deducted for results in the report that cannot be reproduced from submitted code (allowing for variation due to random model initialisation etc.).

Existing models are allowed, provided they are properly referenced. LLM assistance is also allowed. In all cases, the submitted work remains the responsibility of the group.

Some course content has previously been delivered using Jupyter notebooks. This is not a requirement for the project. A notebook may be used during model development or training for documentation or collaboration purposes if desired, but the final inference submission must be a standalone Python file named `inference.py`.

It is sufficient if one group member submits. Make sure that all other group member names are listed in the report and each member has signed (!) the member contribution document.

The following pages contain the questions to be addressed in the report, and the marking criteria.

Question 1: Image Classification for a Smart Fridge (COMP3330 and COMP6380)

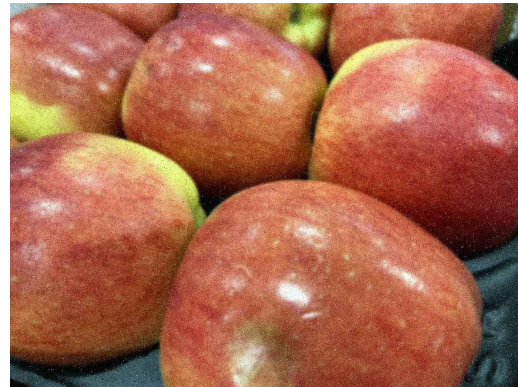
Samsung's smart fridge range uses AI vision and an internal camera. In this assignment, the smart fridge remains the motivating application, but the provided food images are more general and should not be assumed to come from an internal fridge camera perspective.

Your task is to develop a computer vision classifier for 40 food item classes. Example images similar to those used for final evaluation are provided in FoodTest1.zip. A separate hidden dataset, FoodTest2.zip, will not be released. It will use the same naming and structure as FoodTest1.zip and will be used during examination. Your submitted model and inference code will be tested on FoodTest2.zip.

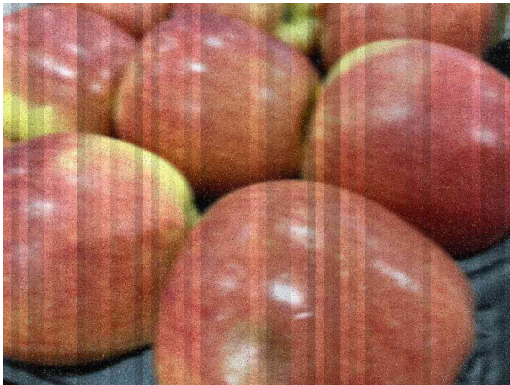
An example food image and three noisy variants are displayed below.



Apples (Clean)



Apples (Noise1)



Apples (Noise2)



Apples (Noise3)

The work for this question should address the following requirements:

- Develop a deep learning model to classify images into the 40 given food item classes. During development you may explore variants such as training from scratch, transfer learning, alternative architectures, hyperparameter choices, preprocessing choices, regularisation strategies, and data handling approaches. Where useful, these variants may be compared and discussed in the report, but the submission should identify and justify one final recommended model.
- FoodTest1.zip contains example data and may be used for training, validation, or both. You may need to obtain additional samples to build a sufficient training dataset.
- Noise robustness should also be considered. Example noisy images are provided on Canvas in NoiseExamples.zip, with subfolders Noise1/, Noise2/, and Noise3/. These contain example images from FoodTest1.zip with varying levels of noise applied. AI-

ternative versions of the hidden FoodTest2.zip dataset may also use varied noise. You may use these examples in any appropriate way to improve robustness.

- Determine which final model is best suited for a smart fridge application, taking predictive performance, computing requirements, power consumption, robustness, and practical deployment considerations into account. The supporting discussion should be documented in your group report, including figures and tables comparing relevant variants or candidate approaches where appropriate, their advantages and disadvantages, and the performances you achieve.
- Prepare separate training and inference scripts. You will need to save your trained model from the training workflow so that it can be loaded and used by the inference script. You may also include other scripts if relevant, such as data processing or preprocessing scripts.
- The final inference component must be a standalone Python file named `inference.py`. The training workflow may be implemented in a script or notebook if desired, but the submitted inference component must be a `.py` file.
- The inference script must be executable from a command line environment as:

```
python inference.py <dataset folder>
```

For example:

```
python inference.py ./FoodTest1
```

The script may optionally support additional command-line arguments such as an alternative model path for testing, but the default behaviour should load the intended submitted model automatically.

- The inference script must load images from the supplied dataset folder and run inference using the submitted model.
- Verify your inference code using FoodTest1. Since FoodTest2 will use the same naming and structure as FoodTest1, your code should generalise to that dataset without requiring changes. Failure to generalise to FoodTest2 may result in lost marks for code.
- The inference script should also print its inference accuracy using the following format:

```
Dataset: <dataset_folder>
Class      Samples  Correct  Accuracy
0_Asparagus  3         2       66.67%
...
39_Cheese   11        9       81.82%

Mean Class Acc: 72.24%
Overall Acc:   78.57%
```

Include all class rows in your version. The mean class accuracy is the average of the accuracy values across classes, while the overall accuracy is the per-sample accuracy.

- Include a README file with clear instructions. If your code requires specific packages and/or versions of packages, include a `requirements.txt` file specifying them.

Question 2: Smart Fridge System Design (COMP6380 only)

Based on the deep learning study in Question 1, propose and discuss a possible design of the AI system for a new smart fridge and take two cases into account:

- a) A fridge system that can connect to the internet.
- b) A smart fridge system that operates in areas without any internet connection or any external communications.

Provide a literature review and compare and discuss the two use cases.

Marking Criteria

Criterion	Marks	Description/Criteria
Question 1	20	Development of the final food classifier for the 40 classes. Marks are based on the overall quality of the approach, including development and justification of the final model, evidence from experiments or variants where appropriate, hyperparameter choices, data handling, preprocessing, use of FoodTest1.zip and any additional data, consideration of noisy inputs and robustness, and suitability for smart-fridge deployment. The report should clearly document the experimental process, results, and interpretation using suitable figures and tables.
Question 2	10	Proposal and comparison of the two systems makes sense. Consideration is given to energy efficiency, computational requirements, maintenance, and build costs. Detail and depth of the proposal and the comparative discussion. Literature review is useful and up-to-date. <i>COMP6380 only.</i>
Format, Writing and Referencing	5	The report should conform to the Springer LNCS format and be clearly structured, well organised, and easy to read. The report length in pages (excluding References/Appendix) should be 6–10 pages for COMP3330 and 8–14 pages for COMP6380. Be aware that any work after the maximum page limit may not be included in marking. Results and findings should be communicated clearly, accurately, and concisely. This includes correctness of grammar and spelling, and appropriate use of references, direct quotes, paraphrasing, and citations. For COMP6380 groups the citations and presentation are expected to be of high professional standard.
Code	5	Style and adherence to the provided guidelines, including clear structure, documentation, reproducibility, and correct separation of training and inference code. The submitted code should execute in the intended format, including command line execution via <code>python inference.py <dataset folder></code> , loading of the intended submitted model by default, printing of the required accuracy summary including the dataset path, and provision of a README file (and requirements.txt if needed). Marks may be deducted for poor documentation, poor structure, failure to execute correctly, or failure to generalise to FoodTest2.zip and possible noisy variants.

Maximum marks: COMP3330 = 30 marks; COMP6380 = 40 marks.